

MIDTERM PROJECT

Deep Learning Optimizers

Comparing Adam, AdamW & RMSprop on the KMNIST dataset

MOTIVATION

Why optimizers matter

Same dataset. Same network. The optimizer alone decides how quickly the model learns — and whether it **generalizes** or simply **memorizes**.

- 01 Implement and tune Adam, RMSprop and AdamW on one fixed network.
- 02 Compare convergence speed, training stability and final accuracy.
- 03 Draw evidence-based conclusions about each optimizer's trade-offs.

1.46

PERCENTAGE POINTS

Spread in final test accuracy between our best and worst optimizer — on identical data and the identical network.

THE DATASET

KMNIST (Kuzushiji-MNIST)



The 10 classes — one cursive hiragana per label

60,000

training images

10,000

test images

28×28

grayscale pixels

10

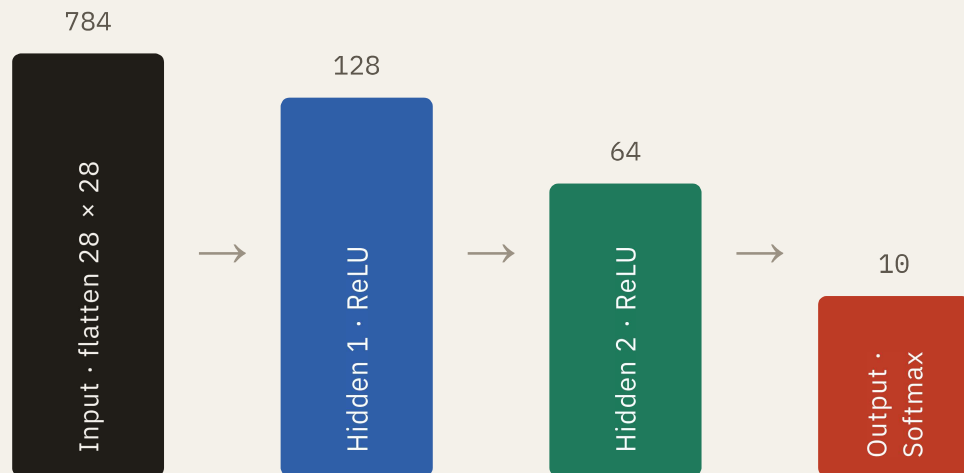
hiragana classes

A drop-in replacement for MNIST, but **harder**: cursive Kuzushiji forms make class boundaries overlap.

THE MODEL

One fixed network for everyone

A fully-connected feedforward classifier — held constant so the optimizer is the only variable.



Loss Cross-Entropy (softmax applied inside) **Fixed** Architecture frozen — differences come only from the optimizer

Three optimizers at a glance

RUCHI

Adam

Adaptive Moment Estimation

Per-parameter learning rates from running estimates of the gradient's **momentum** and **variance**.

We vary $\text{lr} \cdot \beta_1$

KRUTIK

RMSprop

Root Mean Square Propagation

Divides the learning rate by a **running average of squared gradients**, keeping update sizes in check.

We vary $\text{lr} \cdot \alpha$

DILKHUSH

AdamW

Adam + Decoupled Weight Decay

Fixes Adam's L2 flaw by **decoupling weight decay** from the adaptive step for cleaner regularization.

We vary $\text{lr} \cdot \text{weight decay}$

Adam

Adaptive Moments

Combines first-moment **momentum** and second-moment **variance** estimates into a separate adaptive learning rate for every parameter.

DEFAULTS

lr 0.001

β_1 0.9

β_2 0.999

ϵ 1e-8

- Stable training across configurations.
- Moderate learning rates win; lr = 0.01 grows unstable.

TEST ACCURACY

88.69%

Train acc

99.53%

Gen. gap

10.84

best • lr 0.001 • β_1 0.9

RMSprop

Squared-Gradient Scaling

Divides the learning rate by a **running average of squared gradients** (decay α), preventing any single update from growing too large.

DEFAULTS

lr 0.01

α 0.99

ϵ 1e-8

- Fastest convergence; very stable at the right rate.
- lr-sensitive — best at 0.001, below the 0.01 default.

TEST ACCURACY

89.76%

Train acc

99.62%

Gen. gap

9.86

best • lr 0.001 • α 0.99

AdamW

Decoupled Weight Decay

Corrects Adam's L2 flaw by **decoupling weight decay** from the adaptive update, so regularization stays clean and generalization improves.

DEFAULTS

`lr 0.001` `β_1 0.9``wd 0.01`

- Smallest train–test gap of the three.
- wd 0.01 + lr 0.001 best; lr = 0.1 fails entirely.

TEST ACCURACY

90.15%

Train acc

98.20%

Gen. gap

8.05

best · lr 0.001 · wd 0.01

A fair, repeatable protocol

01

Search

≥ 4 configs per optimizer.
Learning rate swept across
two orders of magnitude,
plus one optimizer-specific
parameter.



02

Cross-validate

5-fold cross-validation on
the training set; record
mean ± std validation
accuracy for every
configuration.



03

Select

Keep the single best
configuration per optimizer
— the one with the highest
mean validation accuracy.



04

Train & test

Train the final model on the
full set for 20 epochs, then
report accuracy and loss on
the held-out test set.

Held constant —

seed 42

20 epochs

batch 64

CrossEntropy loss

identical network

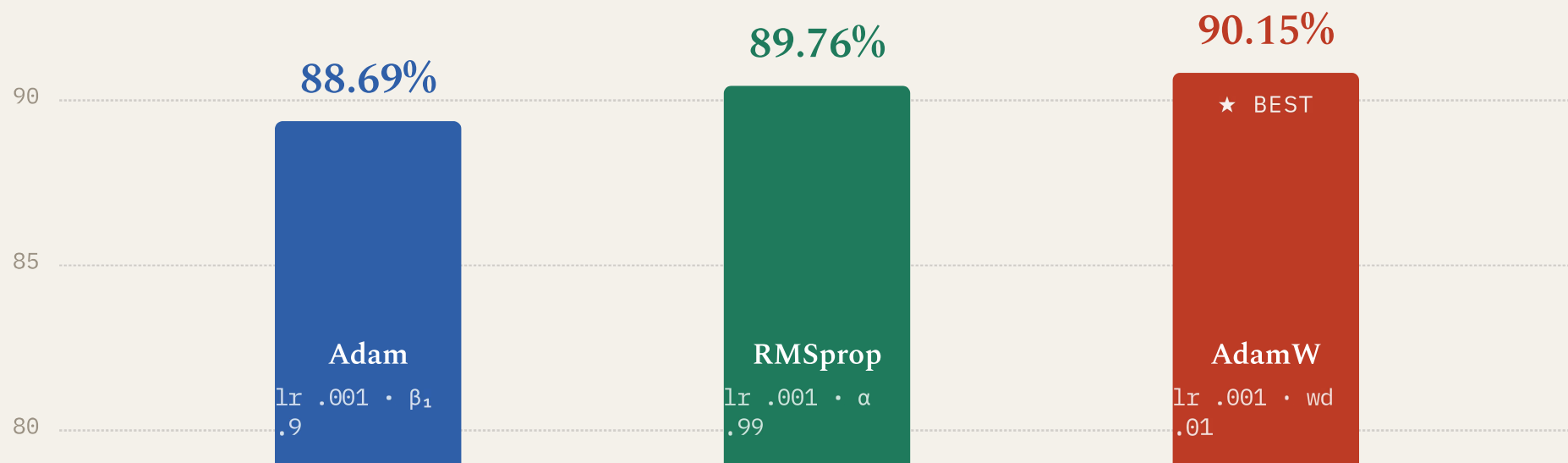
RESULTS

Head-to-head

best configuration of each

METRIC	Adam	RMSprop	AdamW ★
Best configuration	lr .001 · β_1 .9	lr .001 · α .99	lr .001 · wd .01
Test accuracy	88.69%	89.76%	90.15%
Train accuracy	99.53%	99.62%	98.20%
Generalization gap	10.84	9.86	8.05
Convergence	Fast	Fastest	Steady
Best learning rate	0.001	0.001	0.001

Final test accuracy

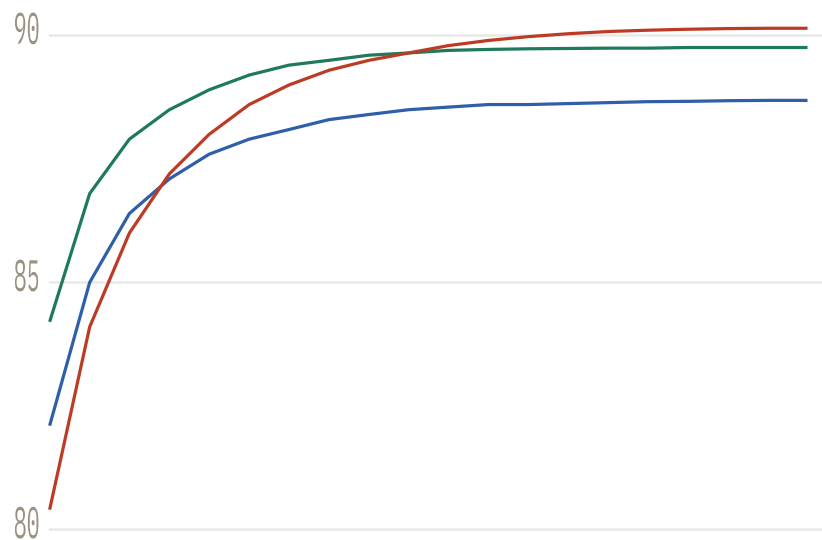


Test accuracy (%) · vertical axis begins at 80% to show the spread

Convergence over 20 epochs

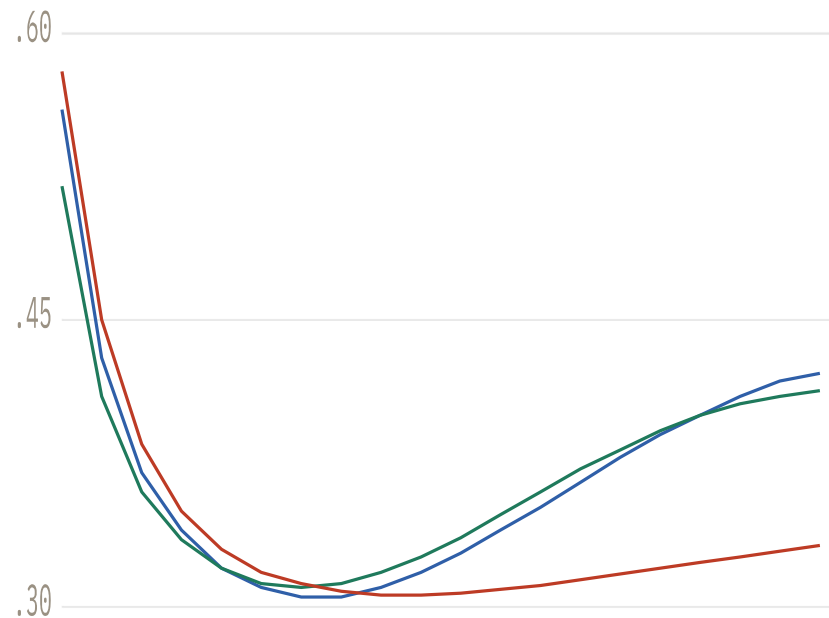
Adam RMSprop AdamW

Test accuracy (%)



Epoch 1 → 20

Test loss



Epoch 1 → 20

DISCUSSION

The generalization gap

Train accuracy minus test accuracy — how much each optimizer memorized rather than learned.

Adam

10.84



RMSprop

9.86



AdamW ★ SMALLEST

8.05



Weight decay closes the gap

AdamW's decoupled weight decay penalizes large weights cleanly — so it generalizes best, even though its train accuracy is the lowest.

Learning rate was the biggest lever

All three peaked at $\text{lr} = 0.001$; a rate of 0.1 was unstable or failed outright — more impactful than any optimizer default.

Choosing an optimizer

Adam

Reliable default

Fast convergence and low tuning effort. The safe first choice when prototyping a new model.

CNNs · Transformers · first attempts

RMSprop

Speed & non-stationary

When training speed matters or the objective is noisy and shifting. Needs a little more lr tuning.

RNNs · real-time · noisy signals

AdamW

When generalization matters · our pick

The modern default when overfitting is a concern. Best test accuracy and smallest gap on KMNIST.

Large models · fine-tuning · production

What we'd do with more time

Limitations

- Only one dataset (KMnist).
- Simple feedforward net, no convolution — accuracy is capped.
- Small search: four configurations per optimizer.
- Single random seed; no learning-rate scheduler.

Future work

- ▶ Evaluate on CIFAR-10 / 100.
- ▶ Convolutional and deeper architectures.
- ▶ Wider search + learning-rate schedules.
- ▶ Multiple seeds to quantify variance.

CONCLUSIONS

What we learned

WINNER ON KMNIST

AdamW

90.15% test acc

Best accuracy and the smallest generalization gap, through decoupled weight decay.

- 1 All three optimizers trained the fixed network successfully on KMNIST.
- 2 RMSprop converged fastest; Adam was the most robust baseline.
- 3 Learning rate, chosen by cross-validation, mattered more than any default.

TEAM & REFERENCES

Who did what

Ruchi	Adam experiments — learning-rate and β_1 tuning.
Dilkhush	AdamW experiments — weight-decay and generalization analysis.
Krutik	RMSprop experiments — α and stability testing.
Isha	Team lead — merged the code into a shared pipeline, documentation and comparison.

REFERENCES · APA

Kingma, D. P., & Ba, J. (2015). *Adam: A method for stochastic optimization*. ICLR.

Loshchilov, I., & Hutter, F. (2019). *Decoupled weight decay regularization*. ICLR.

Tieleman, T., & Hinton, G. (2012). *Lecture 6.5 — RMSprop*. Coursera.

Clanuwat, T., et al. (2018). *Deep learning for classical Japanese literature*. NeurIPS Workshop.

Thank you

Questions?